

PRAKTIKUM SISTEM OPERASI

MODUL 5

Eksplorasi Proses Xinu



Oleh:

Kelvin Ferdinan

2311104009

PROGRAM STUDI S1 REKAYASA PERANGKAT LUNAK

DIREKTORAT KAMPUS PURWOKERTO

UNIVERSITAS TELKOM

2026

JURNAL

1. Soal 1

1. [10 Poin] Jawablah pertanyaan berikut ini:

- a. Berapa banyaknya maksimum proses yang ada pada Xinu?
- b. Berapa maksimal panjang nama suatu proses pada Xinu?
- c. Berapa nilai prioritas awal pada saat proses dibuat?
- d. Ada berapa total state pada Xinu? Sebutkan!

a. Maksimum proses: 8 proses (NPROC = 8).

b. Maksimal panjang nama proses: 16 karakter (PNMLEN = 16).

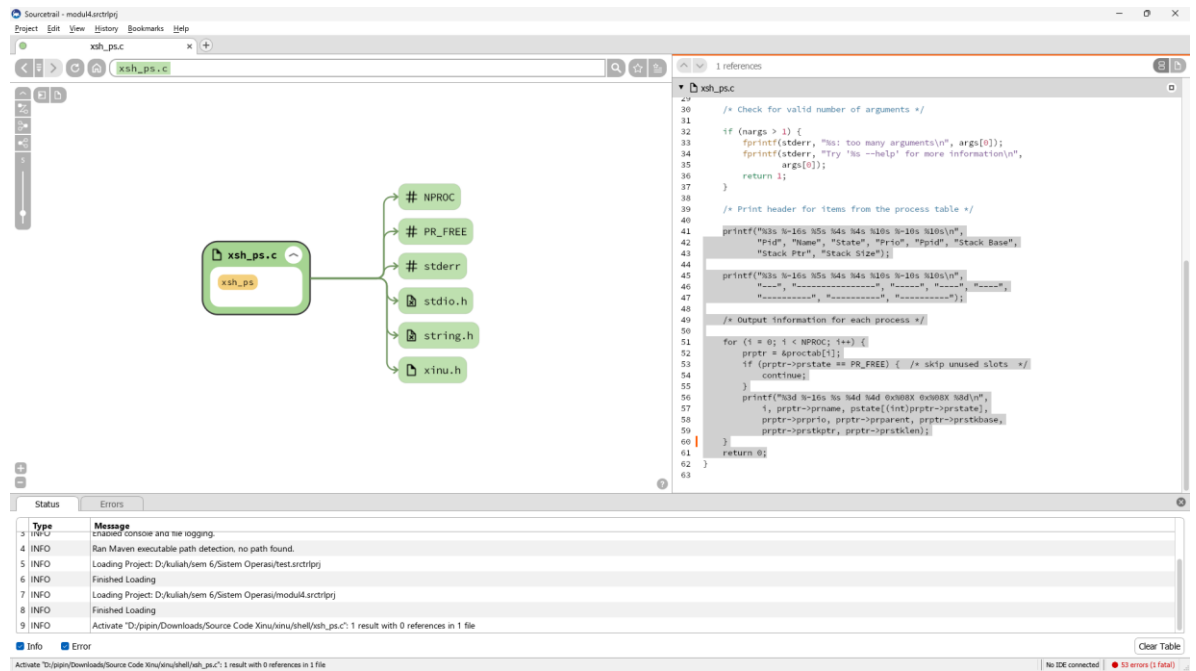
c. Prioritas awal saat proses dibuat: 20 (INITPRIO = 20).

d. Total state proses: 8 state, yaitu:

PR_FREE, PR_CURR, PR_READY, PR_RECV, PR_SLEEP, PR_SUSP,
PR_WAIT, PR_RECTIM.

2. Soal 2

2. [20 Poin] Perintah ps adalah perintah untuk menampilkan statistik process yang berjalan. Source code dari ps tersimpan pada file xsh_ps.c. Carilah file tersebut dan beri komentar pada 20 baris terakhir di source code tersebut!



3.

3. [35 Poin] Ubahlah perintah ps (source code: xsh_ps.c) pada Xinu sehingga menampilkan informasi tambahan berupa kolom yang berisi total message yang ada pada proses seperti gambar di bawah ini:

Pid	Name	State	Prio	Ppid	Stack Base	Stack Ptr	Stack Size	Msg Content
0	prnull	ready	0	0	0x005F0FFC	0x00FFFF04	8192	1 4
1	rdspc	susp	200	0	0x005F8FFC	0x005FBFC8	16384	0 0
2	ipout	wait	500	0	0x005F7FFC	0x005F7F40	8192	0 0
3	netin	wait	500	0	0x005F5FFC	0x005F5EE0	8192	0 0
5	Main process	recv	20	4	0x005E3FFC	0x005E3F64	65536	0 0
6	shell	recv	50	5	0x005D3FFC	0x005D3C7C	8192	0 0
7	ps	curr	20	6	0x005F3FFC	0x005F3DC8	8192	0 0

Kolom Msg adalah banyaknya pesan yang ada dalam proses.

Kolom Content adalah isi dari pesan tersebut.

Langkah pengerjaan:

- Modifikasi source code pada file xsh_ps.c
- Kompilasi ulang Xinu dengan perintah seperti pada modul sebelumnya
- Jalankan Backend VM
- Setelah sistem berjalan, jalankan perintah \$ps. Pastikan hasilnya sesuai dengan contoh output pada gambar yang diberikan.
- Screenshot source kode dan output akhir hasil modifikasi

```

/* Print header for items from the process table */

printf("%3s %-16s %5s %4s %4s %10s %-10s %10s\n",
       "Pid", "Name", "State", "Prio", "Ppid", "Stack Base",
       "Stack Ptr", "Stack Size", "Msg", "Content");

printf("%3s %-16s %5s %4s %4s %10s %-10s %10s\n",
       "----", "-----", "-----", "-----", "-----",
       "-----", "-----", "-----");

/* Output information for each process */

for (i = 0; i < NPROC; i++) {
    prptr = &proctab[i];
    int msg_count = 0;
    uint32 msg_content = 0;

```

4. [35 Poin] Ubahlah perintah uptime pada Xinu sehingga menampilkan lamanya Xinu sejak booting **hanya dalam satuan menit**.

Langkah pengerjaan:

- Modifikasi source code pada file xsh_uptime.c
- Kompilasi ulang Xinu dengan perintah seperti pada modul sebelumnya
- Jalankan Backend VM
- Setelah sistem berjalan, jalankan perintah `$uptime`. Pastikan hasilnya sesuai dengan contoh output yang diinginkan
- Screenshot source kode dan output akhir hasil modifikasi

4.

```

/* Check for valid number of arguments */

if (nargs > 1) {
    fprintf(stderr, "%s: too many arguments\n", args[0]);
    fprintf(stderr, "Try '%s --help' for more information\n",
            args[0]);
    return 1;
}

secs = clktime;          /* Total seconds since boot */
minutes = secs / 60;

kprintf("Uptime %u: menit\n", minutes)

/* Subtract number of whole days */

days = secs/secperday;

```

```
/* Check for valid number of arguments */

if (nargs > 1) {
    fprintf(stderr, "%s: too many arguments\n", args[0]);
    fprintf(stderr, "Try '%s --help' for more information\n",
               args[0]);
    return 1;
}

secs = clktime;          /* Total seconds since boot */
minutes = secs / 60;

kprintf("Uptime %u: menit\n", minutes)

/* Subtract number of whole days */

days = secs/secperday;
```